

Lecture 7: More pointers & dynamic memory

CMPE I20: Computer Organization & Architecture

Olivia Weng

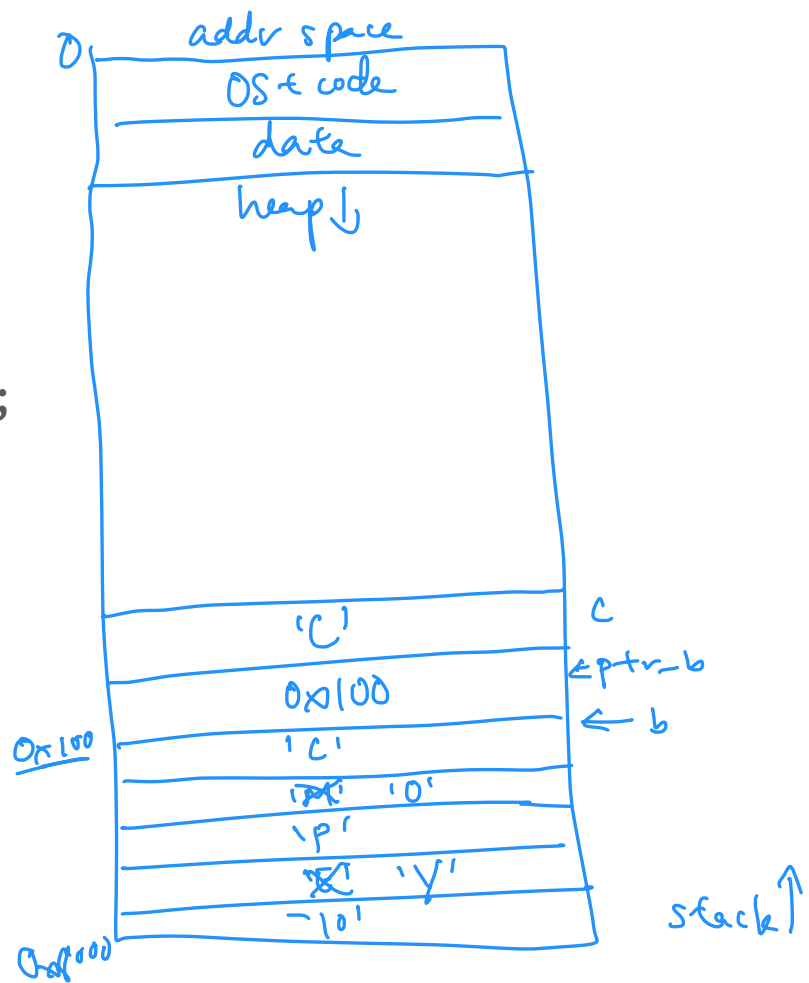
What will be printed?

- Hint: draw the stack!

```
→ char b[] = {'C', 'M', 'P', 'E', '\0'};
   char *ptr_b = b;
→ b[1] = 'O';
→ ptr_b[3] = 'Y';
→ char c = *b;
```

```
printf("Values: %c\n", ptr_b[1]); 'O'
printf("Values: %c\n", b[1]); 'O'
printf("ptr_b = %s\n", ptr_b); COPY
printf("b = %s\n", b); COPY
printf("c = %c\n", c); C
```

Null
terminator
char



Address or value?

```
int num = 42;  
int *a = &num;  
int *b = a;  
int c = *a;
```

- Is this an address (a) or value (v)?
 - a
 - *a
 - c
 - b
 - *b
 - &c

Bonus question: Does `&c == &num`?

- A. v, a, v, v, a, v
- B. v, a, a, a, v, a
- C. a, v, v, a, v, a
- D. a, v, a, a, v, a

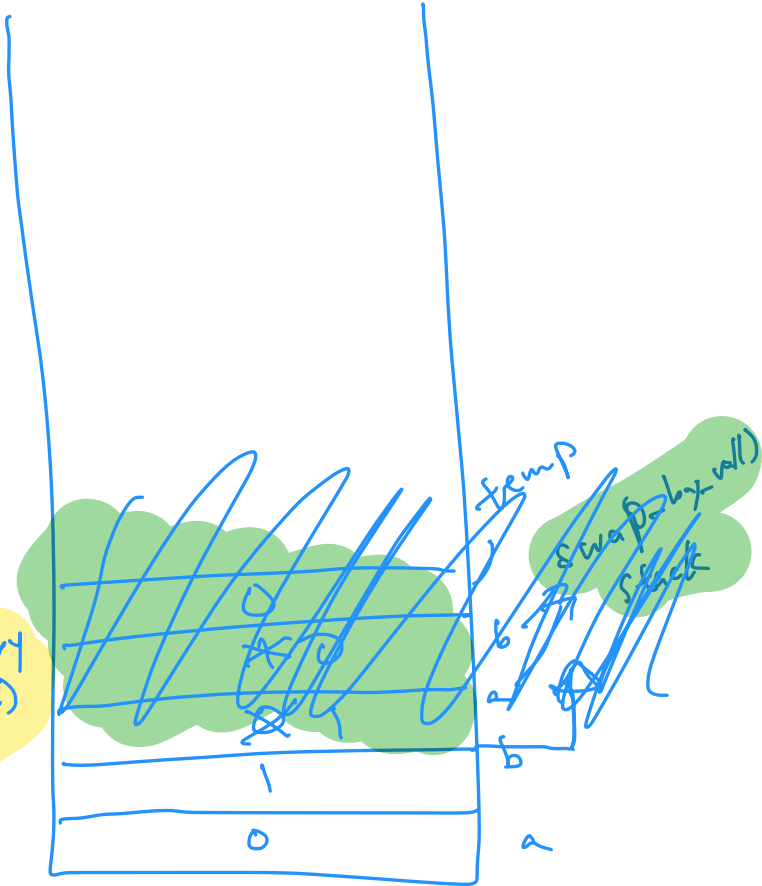
A: Yes
B: No

Demo

- void swap_by_val(int a, int b)
- void swap_by_ptr(int *a, int *b)

void swap_by_val() {
 int temp = a;
 a = b;
 b = temp;
}

swap by ptr
int temp = *a;
*a = *b;
*b = temp;



A pointer can also point to an array?

- An array is a region of memory allocated to a set of values of a specific data type
 - The name of an array corresponds to the address of the first element of an array

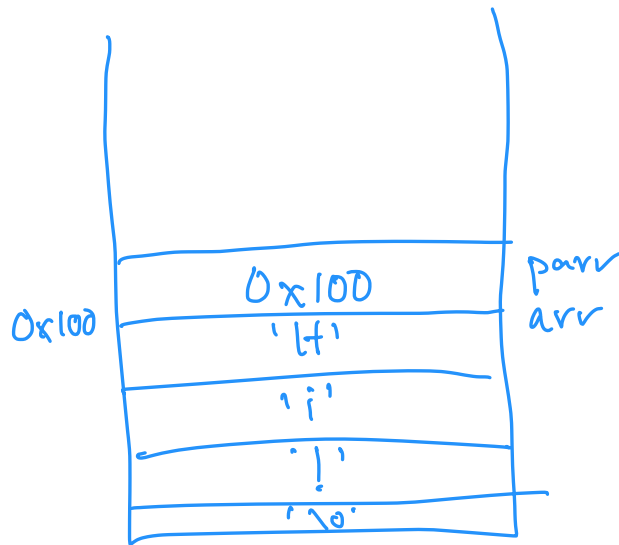
```
char arr[4] = {'H', 'i', '!', '\0'};
```

```
char *parr = arr;
```

```
printf("Values: %c %c\n", arr[1], parr[1]);
```

```
printf("Addresses: %p %p\n", arr, parr);
```

```
assert(arr == parr);
```



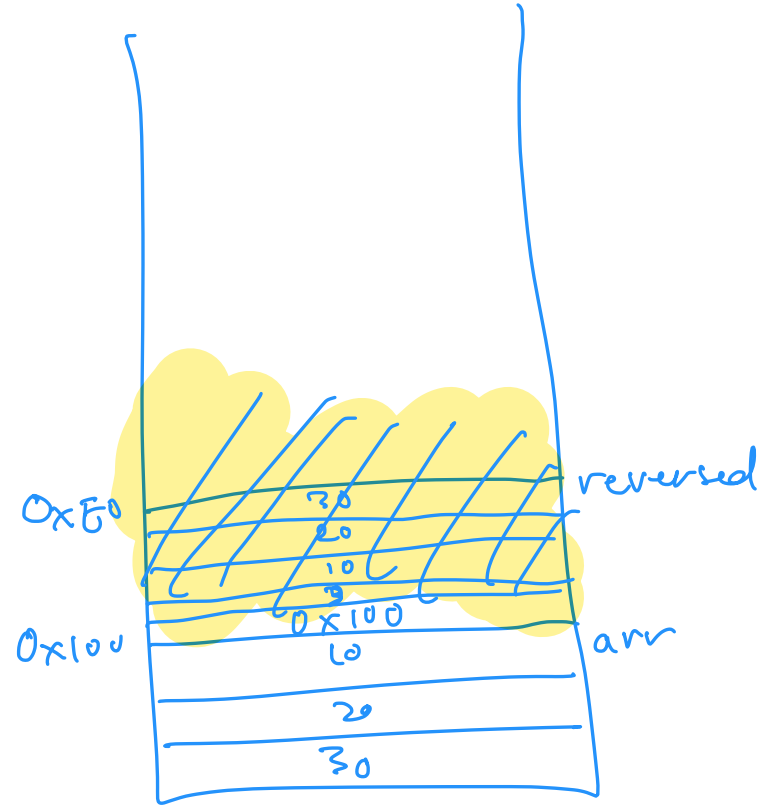
Demo

A: yes

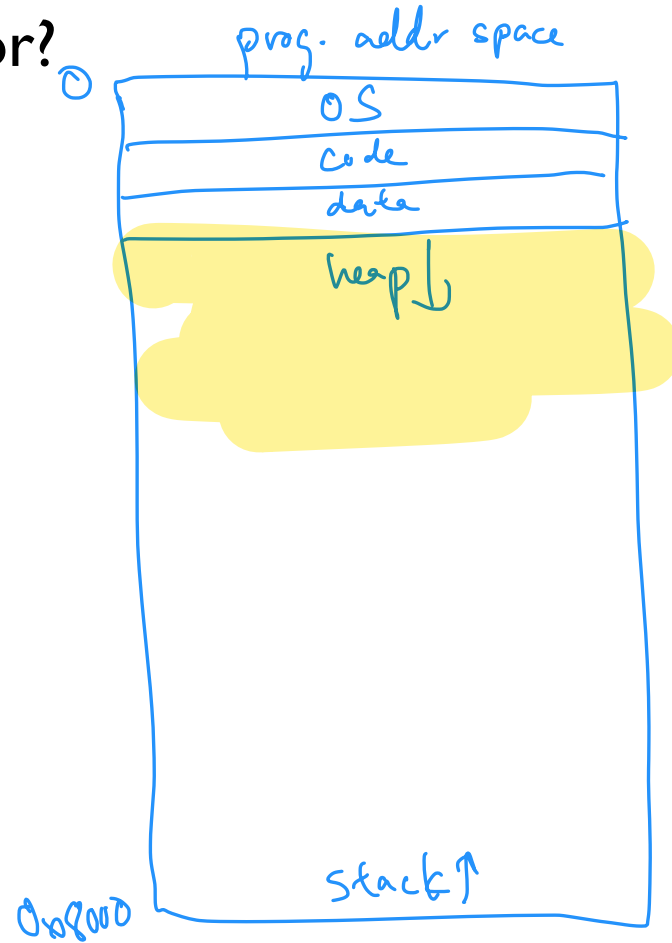
B: no

```
int *reverse(int arr[], int n) {  
    int reversed[n];  
    for (int i = 0; i < n; i++) {  
        reversed[i] = arr[n - i - 1];  
    }  
    int *to_return = reversed;  
    return to_return;  
}
```

```
int main() {  
    int arr[3] = {10, 20, 30};  
    int *reversed = reverse(arr, 3);  
    for (int i = 0; i < 3; i++) {  
        printf("%d\n", reversed[i]);  
    }  
}
```



What is the heap for?

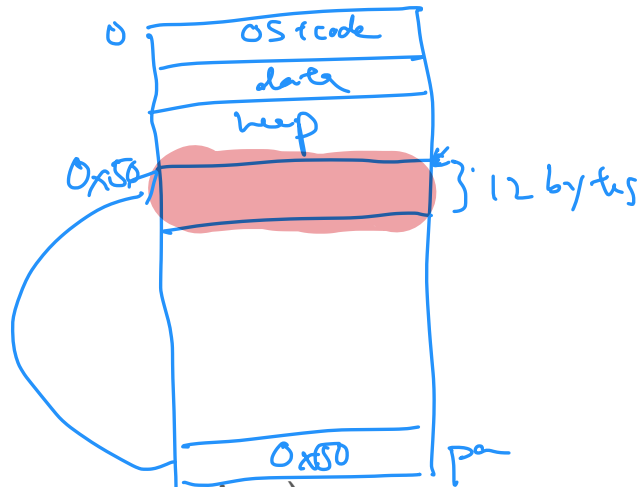


Heap memory

- Heap memory can be accessed from any function as long as we have pointers to it
- What if we want to create an array that persists after a function returns?
 - Use the heap!

How to request memory from the heap?

- `malloc(num_bytes)`
 - stands for "memory allocate"
 - returns the address of the first byte in the memory allocated
 - part of `stdlib.h`
- Example: variable-length array (length of the array changes over time)



```
int *pa = malloc(3 * sizeof(int)); // can do whatever array things I want with pa
```

→ # of bytes of given data type

4 = allocated 12 bytes

pa [o]
pa + i

Demo

- `reverse_fix()`

What happens to heap memory after `malloc()`?

- Heap memory stays there forever so other functions can use it
 - Nice, but what happens when you're done?

We must free() the heap

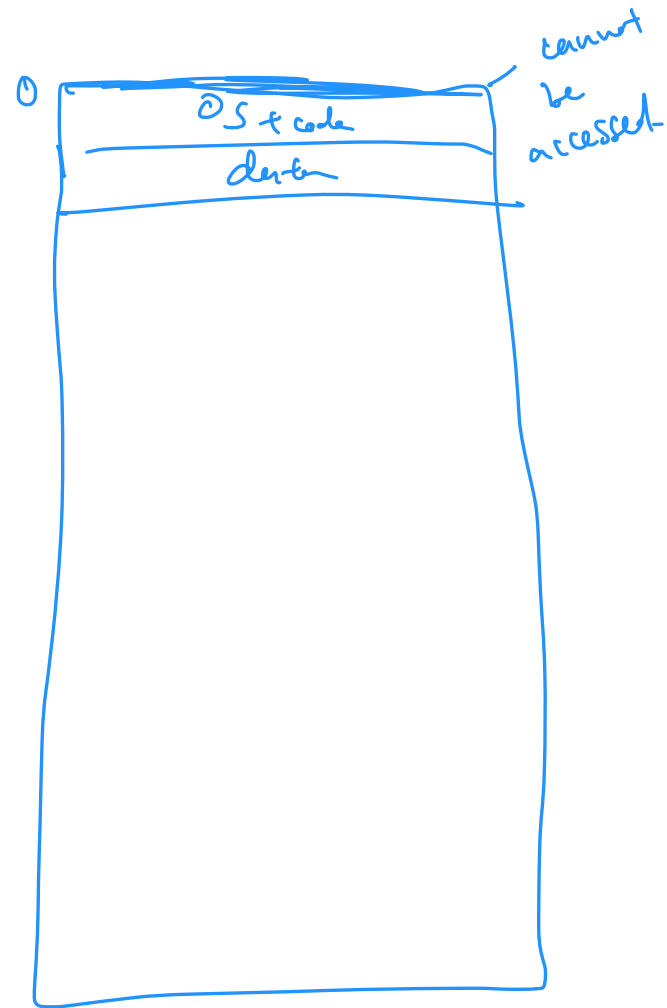
- `free(void *ptr)`
 - Tells the heap to free memory that it allocated at this pointer
 - Now other processes can use it
- What happens if we do not `free()`?
 - **Memory leak!**
 - Performance degradation (slow down the computer)
 - Other processes cannot get the memory they need

What if my pointer points to nothing?

- A pointer points to address 0 — null pointer

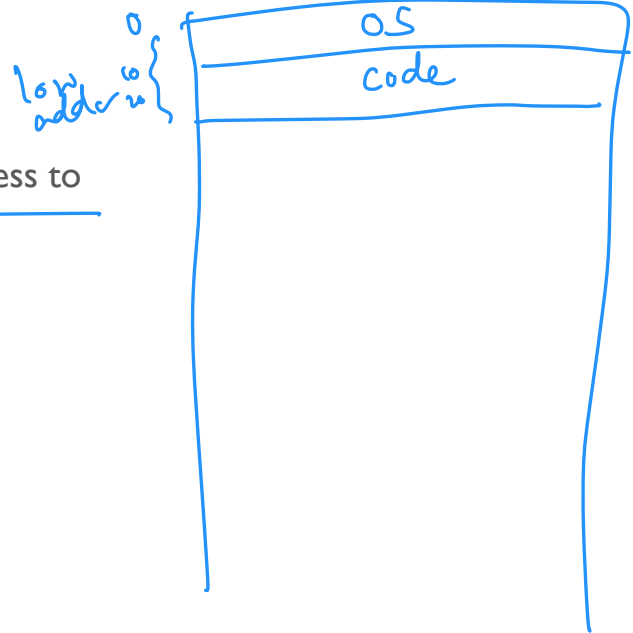
```
int *p = 0x0;
```

```
printf("%d \n", *p); // segmentation fault!
```



What is a segmentation fault?

- **segfault**: when a program tries to access memory it is not allowed to access.
- When does this happen?
 - Dereferencing a null pointer
 - Dereferencing a pointer the program does not have access to



Should I panic when I segfault?

- No! Instead:
 - Look at the pointers you are working with
 - print them with %p
 - If it's 0:
 - likely dereferencing null pointer
 - If it's close to 0:
 - likely dereferencing a pointer you don't have access to
- How to fix?
 - Back trace and see how the pointer's value and/or address was changed